Format-Aware SpMV: Benchmarking CSR, ELLPACK & BCSR with OpenMP Parallelism and AVX2 SIMD

Jerry Ji & Markus Beasley CS 5220 · Applications of Parallel Computers · Cornell University · Spring 2026

COURSE
CS 5220 · Research Showcase

01 MOTIVATION

Sparse Matrix-Vector Multiplication (SpMV) — computing $y = Ax$ — underpins iterative solvers, graph analytics, and scientific simulation. It consistently runs far below peak throughput.

The bottleneck is **memory bandwidth**, not arithmetic: indirect column accesses scatter DRAM reads and destroy cache locality. The storage *format* determines the access pattern and therefore the achievable speed.

> **Question:** Which format wins for each matrix structure — and does AVX2 SIMD actually help?

02 FORMATS STUDIED

CSR
Compressed Sparse Row. Universal baseline. No padding; direct row access.

ELLPACK
Column-major, padded to max row NNZ. Unit-stride SIMD loads when rows are uniform.

BCSR 2×2
Dense 2×2 tile blocks. Reduces index overhead for structured matrices.

BCSR 4×4
4×4 tiles + AVX2 block-row dot product. Best potential for block-dense data.

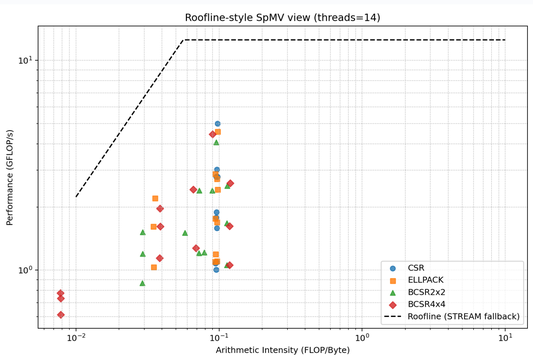
03 MATRIX FAMILIES

12 synthetic matrices, **locked before benchmarking**. 4 families × 3 sizes (S/M/L):

Irregular Graph · FEM Mesh · Block Structured · Banded PDE

- **Irregular graph** — power-law degree, random scatter. 3k–6k rows.
- **FEM mesh** — 2D 9-point stencil. 3k–7k rows.
- **Block structured** — dense 4×4 tiles, periodic. 2k–4k rows.
- **Banded PDE** — bandwidth 3–7. 4k–8k rows.

04 ROOFLINE — AVX2 ON, 14T



All 48 data points land at arithmetic intensity ≪ 1 FLOP/Byte — firmly **bandwidth-bound**. The dashed ceiling (~15 GFLOP/s) is the STREAM peak. CSR reaches ~3.5 GFLOP/s on banded matrices; BCSR4×4 trails furthest left due to zero-padding increasing byte traffic without adding FLOPs.

05 BENCHMARK SETUP

4	3	14	20
formats in C + OpenMP	configurations run	thread counts swept	timed runs / point

1 Generate
Fixed RNG seed. Parameters locked pre-run to avoid cherry-picking.

2 Verify
All formats vs. dense reference. Max error < 10^{-6}.

3 Measure
5 warm-up discarded. Median of 20 runs per (matrix, format, threads).

3 configs: (A) AVX2 on, 1–14T · (B) AVX2 off, 1–14T · (C) AVX2 on, 1–8T

06 RESULTS (CONFIG A, 14 THREADS)

Family	Best Format	GFLOP/s	BCSR worst
Banded PDE	CSR ★	2.96	0.71
FEM Mesh	ELLPACK ★	1.91	0.84
Block Structured	CSR ★	1.82	0.73
Irregular Graph	CSR ★	1.97	0.16

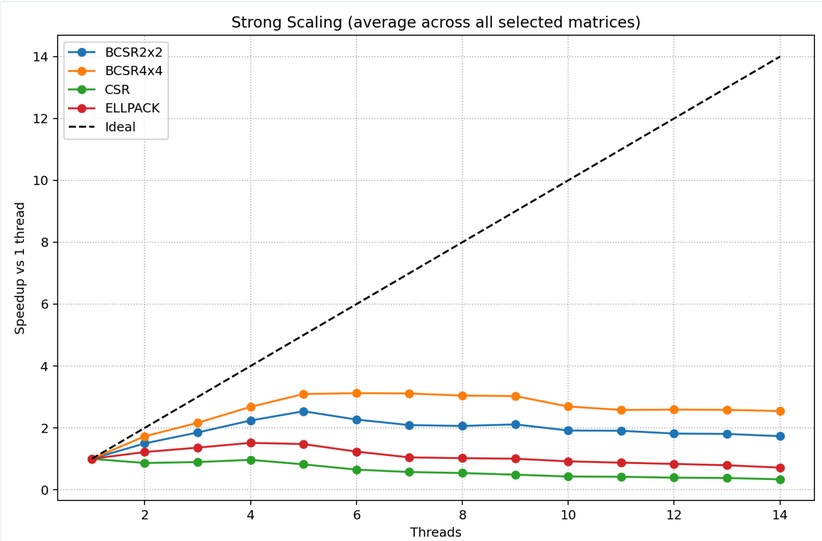
07 SURPRISE: AVX2 CAN HURT

Disabling AVX2 (Config B) often *improved* performance — especially ELLPACK, where gather overhead exceeds the SIMD benefit:

Family	CSR Δ	ELLPACK Δ	BCSR 2×2 Δ	BCSR 4×4 Δ
Banded PDE	−0.5%	+6.3%	−0.7%	−4.5%
FEM Mesh	−3.7%	+2.5%	+1.0%	−4.3%
Block Structured	−4.7%	+0.2%	−3.0%	−4.3%
Irregular Graph	+0.2%	+7.7%	−1.3%	−3.6%

ELLPACK up to +7.7% faster without AVX2. Software gather overhead outweighs 4-wide multiply benefit at these matrix sizes.

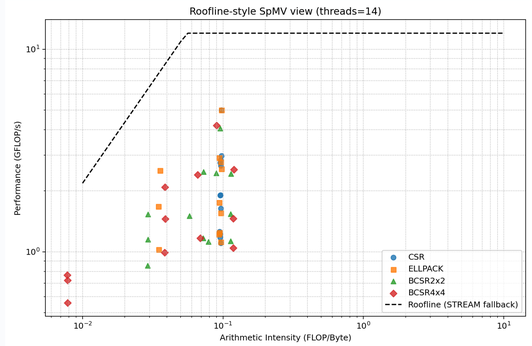
08 STRONG SCALING — AVX2 ON, 1-14T



BCSR4×4 peaks at ~3× near 5 threads then plateaus. **CSR and ELLPACK fall below 1×** — bandwidth contention with many threads actively *hurts* these small matrices. Parallelism only pays off when the problem is large

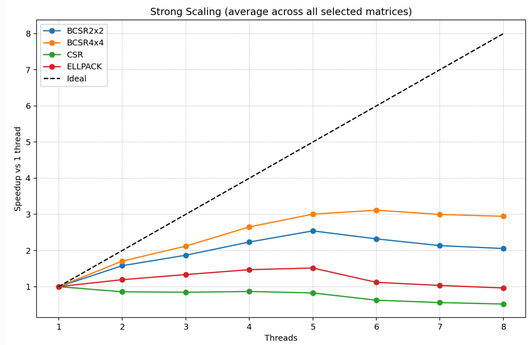
09 AVX2 OFF VS. ON & THREAD COUNT IMPACT

FIG A · ROOFLINE — AVX2 DISABLED, 14 THREADS



With AVX2 disabled, **ELLPACK (orange) shifts noticeably higher** on banded and FEM families. The top ELLPACK point reaches ~3.4 GFLOP/s vs. ~2.8 with AVX2 — a counter-intuitive result explained by software-gather overhead: loading x-vector values for 4-wide SIMD requires serialized indirect loads that cost more than the multiply saves.

FIG B · STRONG SCALING CAPPED AT 8 THREADS (CONFIG C)



At 8 threads, BCSR4×4 scales cleanly to ~3× and ELLPACK on Banded PDE reaches **4.09 GFLOP/s** — 38% better than at 14 threads. This confirms bandwidth saturation: beyond ~5–6 cores, additional threads compete for the same memory bus and performance degrades. Fewer, well-utilized cores win on these small matrices.

PEAK PERFORMANCE
4.09 GFLOP/s — ELLPACK, Banded PDE @ 8 threads

BANDWIDTH SATURATION
8 threads beats 14 threads on every matrix family

AVX2 OVERHEAD
ELLPACK up to +7.7% faster *without* AVX2 SIMD

REMAINING WORK
FUTURE DIRECTIONS

Real-World Matrices (SuiteSparse)
Replace synthetic matrices with SuiteSparse matrices — circuit simulation, social networks, structural FEM — to validate format recommendations at million-row scale on authentic sparsity patterns.

SELL-C-σ Format
Implement Sliced ELLPACK with row-length sorting to reduce padding waste in irregular matrices, potentially recovering the performance gap vs. CSR on graph-like inputs.

Hardware Counter Profiling
Use PAPI or *perf* to measure actual cache miss rates, DRAM bandwidth, and SIMD utilization — replacing STREAM-estimated roofline ceilings with measured data.

Large-Matrix Scaling Study
Scale to millions of rows to observe favorable strong-scaling regimes. Add confidence intervals and parallel efficiency curves to rigorously compare formats in the final report.

GPU Acceleration (CUDA)
Port all four format kernels to CUDA to exploit GPU memory bandwidth. Compare format rankings on GPU — ELLPACK's column-major layout may align well with warp-level coalescing.

Auto Format Selector
Build a lightweight classifier using sparsity pattern features (avg row NNZ, CV, block density) to predict the best-performing format before benchmarking — zero-overhead format dispatch.

Mixed-Precision SpMV
Store matrix values in fp32 while accumulating in fp64. Halves bandwidth for value arrays — potentially recovering the bandwidth-bound gap at no accuracy cost for iterative solvers with residual correction.